LCD 컨트롤러와 충전기기 간 통신 프로토콜

2017년 3월

1. MODBUS PROTOCOL (ASCII DATA 통신)

1) 전송자료의 기본구조

①STX	②고유번호	③명령어(FUNCTION)	④자료(DATA)	⑤CRC	⑥CR
(1)	(2)	(2)	(n * 2)	(4)	(1)

※ ( ) : Byte수

※ ASCII Mode 에서 사용되는 문자는 "0" ~ "9", "A" ~ "F" 입니다.

① STX
- 바이트 수 : 1바이트
- 내용 : 전송자료 프레임의 시작코드(16진수로 '0x02')

② 고유번호
- 바이트 수 : 2바이트
- 내용 : 장비의 고유번호

③ 명령어(FUNCTION)
- 바이트 수 : 2바이트
- 내용 : 명령어
  · "2. 명령어"를 참조

④ 자료(DATA)
- 바이트 수 : 2바이트
- 내용 : 명령어에 해당하는 자료
  · "3. 자료"를 참조

⑤ CRC
- 바이트 수 : 4바이트
- 내용 : CRC-16-CCITT 알고리즘에 의한 CRC코드
  · "4. 자료검증 Byte 계산 방법"을 참조
  ※ CRC-16-CCITT 계산을 위한 자료는 "STX" 이후부터 "CRC" 전까지의 자료로 계산합니다.

⑥ CR
- 바이트 수 : 1바이트
- 내용 : 전송자료 프레임의 종료코드(16진수로 '0x0D')

2. 명령어

명령어 코드	내용	Hexa Code
03	자료 읽기	0x03
06	하나의 파라미터(자료) 저장	0x06
16	다수의 파라미터(자료) 저장	0x10

① '03' : Read Holding Register(0x03)

- 대상이 되는 Slave Device 의 파라메타 값을 읽어오는 Code
- 하나의 파라메타 또는 다수의 연속된 파라메타 값을 읽어 올 수 있습니다.

※ Register 는 파라메타의 차례(400 ~ 4xx)이며, 이에 대응하는 Data Address 는 "400 ~ 4xx" 입니다. ("5. 명령어 예제" 참조)

② '06' : Write Single Register(0x06)

- 대상이 되는 Slave Device 에 특정 파라메타 값을 Setting 하는 Code
- 특정한 하나의 파라메타를 지정하여 값을 Setting 합니다.

※ Register 는 파라메타의 차례(200 ~ 2xx)이며, 이에 대응하는 Data Address 는 "200 ~ 2xx" 입니다. ("5. 명령어 예제" 참조)

③ '16' : Write Multiple Register(0x10)

- 대상이 되는 Slave Device 에 여러 파라메타 값을 Setting 하는 Code
- 연속적으로 나열된(Address 순서에서) 파라메타 값을 Setting 합니다.

※ Register 는 파라메타의 차례(400 ~ 4xx)이며, 이에 대응하는 Data Address 는 "200 ~ 2xx" 입니다. ("5. 명령어 예제" 참조)

3. 자료(DATA)

1) 자료 구성

① Register 의 Address, 개수, data 등으로 구성

② 각 명령어에 따라 구성이 달라집니다

2) 값 구성 및 처리

① 모든 Register 의 data 표시는 4바이트 또는 8바이트로 구성

② Data Type 이 Integer 형인 경우 (4바이트)
- 부호가 없는 data는 '0~65535' 까지, 부호가 있는 data는 '-32768 ~ 32767' 처리
- '50000 (dec)' 에 대한 data 구성 예는 아래와 같습니다.

계산값)

Reg. Value L0-H	Reg. Value L0-L
0xC3	0x50

ASCII 표시)

C	3	5	0

③ Data Type이 Word 형인 경우 (8바이트)
- Register 의 Address를 연속적으로 2개를 사용
- 부호가 없는 data는 '0~4294967296' 까지,
  부호가 있는 data는 '-2147483648 ~ 2147483647' 처리
- '500000 (dec)' 에 대한 data 구성 예는 아래와 같습니다.

계산값)

Reg. Value L0-H	Reg. Value L0-L	Reg. Value L1-H	Reg. Value L1-L
0x00	0x07	0xA1	0x20

ASCII 표시)

0	0	0	7	A	1	2	0

④ 측정값이 소수점인 경우 계산 방법

- 유효 소수점 개수 만큼 10을 곱하여 정수로 변환

예) Integer 형이고 값이 1.2 인 경우

유효 소수점 자리수가 1인 경우 => 1.2 에 10을 곱하여 12로 계산
ASCII 표시값은 "000C"

유효 소수점 자리수가 2인 경우 => 1.2 에 100을 곱하여 120으로 계산
ASCII 표시값은 "0078"

4. 자료검증 Byte 계산 방법

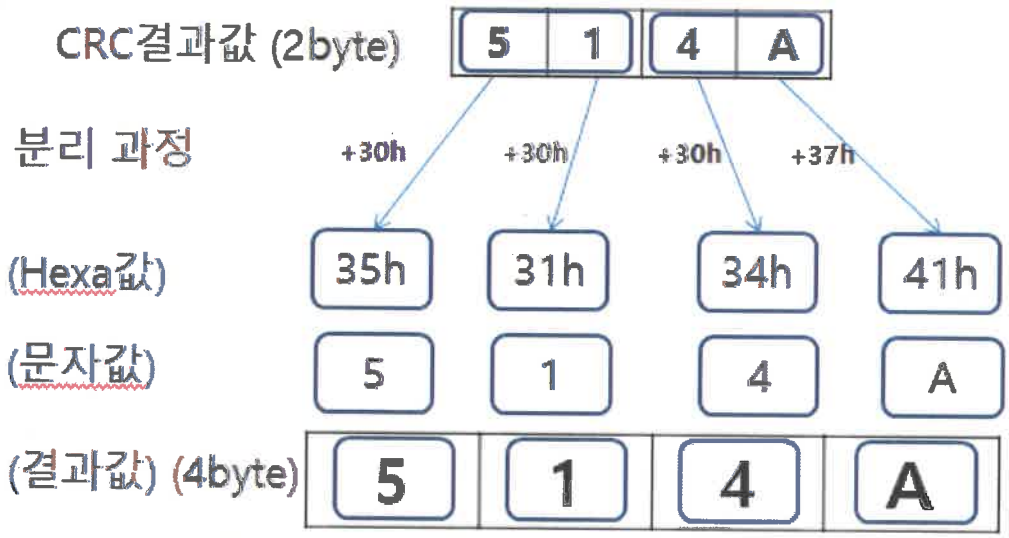
송신자료 Record를 CRC-16-CCITT으로 CRC값을 생성하여, 생성된 총 16bit를 각각 4bit로 분리하여 문자열로 생성한 값을 취한다.

◦ CRC-16-CCITT 알고리즘
참조 : CRC(HDLC standard ISO/IEC 13239(2002) Annex A

◦ CRC-16-CCITT 결과 값 전송자료로 변환 방식

1. 16bit로 출력된 결과를 각각 4bit씩 분리함
   예) 분리값1 = ((crc16값 & 0xF000) >> 16)
   분리값2 = ((crc16값 & 0x0F00) >> 8)
   분리값3 = ((crc16값 & 0x00F0) >> 4)
   분리값4 = (crc16값 & 0x000F)

2. 각 4bit로 분리된 각각의 분리값을 문자로 변환
   단, 문자로 변환 시 문자의 십진수값이 10 미만일 경우 0x30을 더하고, 10 이상일 경우 0x37을 더한다.
   예) if ((int)분리값x < 10)) 문자x = (분리값x + 0x30)
   else 문자x = (분리값x + 0x37)



▶ 1byte 분리 시 0 ~ 9 사이의 값은 +30h, A ~ F 사이의 값은 +37h를 함

◦ CRC-16-CCITT 조건
- 다항식 방향 : 정방향
- 체크썸 크기 : 16 비트
- 생성 다항식 : $X^{16}+X^{12}+X^{5}+1$
- 제 수 : 0x1021
- 나머지 초기값 : 0xFFFF
- 최종 XOR값 : 0x0000

※ 주의사항) **본 계산법은 CRC-16-CCITT의 알고리즘만 사용하였으며, 위의 조건에 맞게 계산하여야 함**

◦ CRC-16-CCITT 소스 예시 (C 언어)

```
typedef unsigned short width;
width crcTable[256];

#define POLYNOMIAL 0x1021
#define INITIAL_REMAINDER 0xFFFF
#define FINAL_XOR_VALUE 0x0000
#define WIDTH (8 * sizeof(width))
#define TOPBIT (1 << (WIDTH - 1))

void initCRC(void)
{
        unsigned short remainder;
        unsigned short dividend;
        int bit;

        for (dividend = 0; dividend < 256; dividend++)
        {
                remainder = dividend << (WIDTH - 8);

                for (bit = 0; bit < 8; bit++)
                {
                        if (remainder & TOPBIT)
                        remainder = (remainder << 1) ^ POLYNOMIAL;
                        else
                        remainder = remainder << 1;
                }

                crcTable[dividend] = remainder;
        }
}

unsigned short calcurateCRC(unsigned char *message, unsigned int nBytes)
{
        unsigned int offset;
        unsigned char byte;
        unsigned short remainder = INITIAL_REMAINDER;

        for (offset = 0; offset < nBytes; offset++)
        {
                byte = (remainder >> (WIDTH - 8)) ^ message[offset];
                remainder = crcTable[byte] ^ (remainder << 8);
        }

        return (remainder ^ FINAL_XOR_VALUE);
}

void MakeCRC16(const char *sData, int iDataLen)
{
        unsigned short crc16;

        initCRC();

        crc16 = calcurateCRC(sData, iDataLen);

        return;
}
```

5. 명령어 예제

① '03' : Read Holding Register(0x03) 사용 예

- Slave Device(ID : 1)의 Register 400 ~ 401 의 값을 읽어오는 경우

요청)

STX (1)	고유번호 (2)		명령어 (2)		Adrress (4)				자료개수 (4)				CRC (4)				CR (1)
	0	1	0	3	0	1	9	0	0	0	0	2	B	8	A	2	

응답)

STX (1)	고유번호 (2)		명령어 (2)		전송 바이트수 (4)				Adrress 400의 값 (4)				Adrress 401의 값 (4)				CRC (4)				CR (1)
	0	1	0	3	0	0	0	8	C	3	5	0	0	7	3	1	C	E	4	7	

② '06' : Write Single Register(0x06) 사용 예

- Slave Device(ID : 1)의 Register 2(Addr : 201)에 '3' 을 setting 하는 경우

요청)

STX (1)	고유번호 (2)		명령어 (2)		Adrress (4)				저장값 (4)				CRC (4)				CR (1)
	0	1	0	6	0	0	C	9	0	0	0	3	0	C	C	C	

응답)

STX (1)	고유번호 (2)		명령어 (2)		Adrress (4)				저장값 (4)				CRC (4)				CR (1)
	0	1	0	6	0	0	C	9	0	0	0	3	0	C	C	C	

※ 명령어 '06' 의 경우 요청전문과 동일한 Frame 으로 응답합니다.
(정상적인 Setting 후)

③ '16' : Write Multiple Register(0x10) 사용 예

- Slave Device(ID : 1)의 Register 2(Addr : 201) 부터 2개의 register 에 각각 '10', '258' 을 setting 하는 경우

요청)

STX (1)	고유번호 (2)		명령어 (2)		Adrress (4)				자료개수 (4)				전송 바이트수 (4)			
	0	1	1	0	0	0	C	9	0	0	0	2	0	0	0	8

자료 #1 (4)				자료 #2 (4)				CRC (4)				CR (1)
0	0	0	A	0	1	0	2	4	C	C	A	

응답)

STX (1)	고유번호 (2)		명령어 (2)		Adrress (4)				자료개수 (4)				CRC (4)				CR (1)
	0	1	1	0	0	0	C	9	0	0	0	2	2	9	2	0	

7. Adrress Map

1) 자료 저장 (LCD 컨트롤러 -> 충전기기)

Address	Name	Bit Name	Bit	Description	factor	unit	Size
200	LCD 컨트롤러 Status	Standby	0	기본 상태 = 1			2 Byte
		Ready	1	충전기에서 Ready 받은 후 = 1			
		Start	2	인증 완료 후 = 1			
		Stop	3	사용자 종료시 = 1			
		Run	4	충전중 = 1			
		Finish	5	충전완료 중 = 1			
		Fault	6	LCD 컨트롤러 Error 발생시 = 1			
			7				
			8				
			9				
			10				
			11				
			12				
		Reset	13	1 : 충전기기 초기화			
		외부등 On/Off	14	1 : On, 0 : off			
		ChargerUpdate	15	1 : update mode, 0 : normal mode			
201	Charging Type			CHAdeMO : 1 COMBO : 2 AC : 3			2 Byte
202	Run Count			Communication RunCount			2 Byte
203	Fault Code			LCD 컨트롤러 Fault Code ex) 결제기 Error, 모뎀 Error 등			2 Byte
210	Version			ex) ver : 1.0 (High Byte : 1, Low Byte : 0)			2 Byte
211	Config	option #1	0	Manufacturer Option			2 Byte
		option #2	1				
		option #3	2				
		option #4	3				
		option #5	4				
		option #6	5				
		option #7	6				
		option #8	7				
		option #9	8				
		option #10	9				
		option #11	10				
		option #12	11				
		option #13	12				
		option #14	13				
		option #15	14				
		option #16	15				

2) 자료 읽기 (LCD 컨트롤러 <- 충전기기)

Address	Name	Bit Name	Bit	Description	factor	unit	Size
400	Charger Status	Ready	0	충전 준비 완료 = 1			2 Byte
		Plug Check	1	커넥터 체크 중 = 1			
		Run	2	충전 중 = 1			
		Finish	3	충전 완료 중 = 1			
		Fault	4	충전기 Error 발생 시 = 1			
			5				
			6				
			7				
		Connector Status	8	커넥터 연결 = 1, 미연결 = 0			
		Connector Lock	9	커넥터 잠금 = 1, 풀림 = 0			
			10				
			11				
			12				
			13				
			14				
			15				
401	Voltage			Output Voltage	0.1	V	2 Byte
402	Current			Output Current	0.1	A	2 Byte
403	Remaing Charging Time				1	min	2 Byte
404	State of charge				1	%	2 Byte
405	Target Voltage			EV Target Voltage	0.1		2 Byte
406	Target Current			EV Target Current	0.1		2 Byte
407	Power Meter			적산유효전력량		Wh	4 Byte
408							
409	Run Count			Communication RunCount			2 Byte
410	Fault Code			Charger Fault Code			2 Byte
411	Version			ex) ver : 1.0 (High Byte : 1, Low Byte : 0)			2 Byte
412	Charging Power Meter			충전 된 전력량			4 Byte
414	Manufacturer Option		0	#1 파워모듈 상태			4 Byte
			1				
			2	#2 파워모듈 상태			
			3				
				...			
			28	#15 파워모듈 상태			
			29				
			30	#16 파워모듈 상태			
			31				
416	Connector Box Info		0	CHAdeMO Connector 박스내 정위치=1,기타=0			2 Byte
			1	CHAdeMO Connector 박스 잠금=1,기타=0			
			2				
			3	COMBO Connector 박스내 정위치=1,기타=0			
			4	COMBO Connector 박스 잠금=1,기타=0			
			5				
			6	AC Connector 박스내 정위치=1,기타=0			
			7	AC Connector 박스 잠금=1,기타=0			
				...			
			15				